# Detecting AI-Generated Text

A Draft Report on Detection and Attribution Using the RAID Benchmark

GSBS 545, Advanced Machine Learning for Business Analytics | Chris Li and Trevor Jhou

## 1. Problem statement

Tools that generate fluent text on demand are now widely available, which creates a practical problem for anyone who needs to trust the origin of a document: educators grading written work, publishers screening submissions, platforms moderating content, and firms verifying that customer-facing copy was written by the people they paid. This project addresses two questions a business would care about.

The first is detection: given a piece of text, can we tell whether a human or a machine wrote it? This supports a simple yes/no gate. The second is attribution: if a machine wrote it, can we tell which family of model produced it? This supports forensic and policy work, such as identifying which tool is being used to mass-produce content. Both are framed as supervised classification problems and evaluated on held-out data the models never saw during training.

**Scope of this draft.** These are initial findings from a 1,000-prompt subsample chosen for fast iteration, with models run at default or lightly-set hyperparameters. The numbers below should be read as a floor rather than a tuned ceiling. The planned work in Section 10, which includes expanding to the full dataset, systematic hyperparameter tuning, and standardizing the neural network inputs, is expected to raise them.

## 2. Data description

The analysis uses RAID, a public benchmark built specifically for AI-text-detection research. It pairs human-written documents with machine-generated versions of the same prompts across many text types and many generator models. After removing the deliberately adversarial, attacked versions of texts, which are reserved for a separate robustness question, the working corpus contains roughly 467,985 documents drawn from about 13,371 distinct source prompts.

Each document carries useful structure: the human or machine model that produced it, the domain it belongs to (eight categories such as news, recipes, reviews, and academic abstracts), the decoding strategy used to generate it, and a source identifier linking every machine version back to the original human prompt it was based on. That source identifier is central to doing this analysis correctly, for reasons explained in Section 4. For a draft that iterates quickly, the analysis works from a stratified subsample of 1,000 source prompts (about 35,000 documents), which preserves the domain and model diversity of the full set while keeping training times manageable. Scaling to the full corpus is a mechanical change for the final version.

## 3. Exploratory analysis

Three properties of the data shaped every modeling decision.

**Severe class imbalance in the raw form.** Each human prompt spawns roughly thirty machine-generated counterparts, one per model and decoding combination. A detector trained on the

raw mix could score about 97% accuracy by always guessing “machine,” while being useless in practice. This single property dictates both how we sample and how we measure success.

**Topic and length travel with the label.** Because every machine document is generated from a human prompt on the same subject, topic is shared across the human/machine boundary, which is helpful. But generated text can differ systematically in length, and a model can cheat by learning “long equals machine” rather than learning anything about authorship. We control for this when building the balanced sets.

**Difficulty varies by domain.** Casual, formulaic text such as reviews and recipes tends to be easier to separate than dense, edited prose such as news and abstracts. A single headline accuracy number hides this, so a per-domain breakdown is part of the planned analysis for the final report.

## 4. Data preparation and feature engineering

**Cleaning.** We drop the adversarial rows so the draft measures clean detection rather than robustness to evasion, and we reduce to one human document per source.

**Balancing.** For detection we build a balanced set of one human and one machine document per source, giving a 1:1 split of 1,000 versus 1,000. This removes the degenerate-accuracy trap above and lets accuracy and balanced accuracy be read on the same scale. For attribution we take one document per source per generator across the eleven machine models, giving 11,000 documents.

**Leakage-safe splitting, the key methodological point.** Because many documents trace back to a single human prompt, an ordinary random train/test split would scatter near-identical, same-topic documents across both sides, and the model would post inflated scores by recognizing topics it had already seen. We prevent this by splitting on the source identifier, so every document descended from a given prompt stays entirely on one side of the split. This is the difference between a number that looks good and a number that means something.

**Text features.** Documents are converted to numbers using TF-IDF over word unigrams and bigrams, with sublinear term-frequency scaling, a minimum document frequency to drop noise, and a capped vocabulary. This representation suits authorship problems because it captures the function-word patterns, phrasing habits, and punctuation tendencies that distinguish human from machine writing. For the neural network, the high-dimensional sparse features are compressed to a 300-dimension dense representation, since neural nets train more stably on dense input.

**Evidence that feature choices matter.** In an earlier configuration, raising the TF-IDF vocabulary cap from 10,000 to 20,000 terms improved the gradient-boosting detector’s balanced accuracy from 0.8125 to about 0.83, a concrete sign that a richer vocabulary lets the tree model capture more stylistic signal. Character-level n-grams, which catch spacing and tokenization artifacts that word features miss, are the next planned feature addition.

## 5. Modeling approach

We implement three model families so we can compare a simple linear baseline, a strong non-neural ensemble, and a neural network:

- **Logistic Regression** on the TF-IDF features, a transparent linear baseline that sets the bar.
- **LightGBM** (gradient-boosted decision trees), the workhorse for high-dimensional sparse text. Note that this model is itself an ensemble, combining hundreds of weak trees.
- **Keras MLP** (a feed-forward neural network) with two hidden layers, dropout, and L2 regularization, early stopping on validation loss, trained on the dense compressed features. This is the neural-versus-non-neural comparison the project centers on.

All three families are applied to detection. The gradient-boosting and neural models are applied to attribution across the eleven generators.

## 6. Evaluation metrics

**Balanced accuracy is the primary metric.** It averages the model's accuracy on each class separately, so it cannot be gamed by favoring a majority class. Given the imbalance described in Section 3, plain accuracy would mislead. On the deliberately balanced detection set, accuracy and balanced accuracy coincide, which keeps the comparison clean. For attribution we also read the per-class report and confusion matrix, because the useful question is not just overall accuracy but which generators get confused with which.

## 7. Results

Balanced accuracy on the held-out test set, using the leakage-safe grouped split on the draft subsample:

| Task | Model | Balanced accuracy |
|---|---|---|
| Detection (human vs machine) | Logistic Regression | 0.800 |
| Detection (human vs machine) | LightGBM | 0.8125 |
| Detection (human vs machine) | Keras MLP | 0.500 (failed to learn) |
| Attribution (11 generators) | LightGBM | 0.481 |
| Attribution (11 generators) | Keras MLP | 0.494 |

For attribution, a model guessing at random across eleven generators would score about 0.091, so both models at roughly 0.48 are performing about five times better than chance. In the per-class results, GPT-4 was the easiest generator to identify (F1 of 0.63), followed by LLaMA-chat, MPT, and ChatGPT, while the Cohere and Mistral families were the hardest and were most often confused with one another. GPT-2 had high recall but low precision, meaning the model over-assigned text to it.

## 8. Comparison and model behavior

**Detection.** The gradient-boosting ensemble edged out the linear baseline (0.8125 versus 0.800), the expected ordering, since boosting captures non-linear interactions among stylistic features that a linear model cannot. The striking result is the neural network: it scored exactly 0.500, which on a balanced two-class problem is no better than a coin flip. The network collapsed to predicting a single class and

failed to learn the task.

This failure is informative rather than a dead end. The most likely cause is that the 300-dimension compressed features were fed to the network without standardization, against the usual convention of scaling inputs before training, and on only 1,600 training documents the optimizer settled into a trivial all-one-class solution. The same network architecture did learn a non-trivial solution on the eleven-class attribution task (0.494, just ahead of LightGBM at 0.481), which confirms the code is sound and points the blame at the binary setup specifically. Standardizing the inputs and tuning the learning rate and architecture is the clear fix for the final version.

**Attribution.** Here the neural network and the boosting model were effectively tied, both near 0.48 and far above chance. That the neural network matched a strong tree ensemble on the harder eleven-class problem, yet failed on the easier binary one, is a useful reminder that model behavior depends as much on the training setup and input scaling as on the architecture itself.

**Complexity versus performance.** The tradeoff currently favors the boosting model. It trains in seconds on this subsample, needs no GPU, leads on detection, and ties on attribution, while the neural network costs more to train and tune and, as configured, did not justify that cost.

## 9. Conclusions and recommendations

Detecting machine-written text at roughly 0.80 to 0.81 balanced accuracy is achievable with standard text features and a gradient-boosting ensemble, even across eight content domains and eleven different generators, and even under a strict leakage-safe evaluation. That the number is not near-perfect is itself informative: this is a genuinely hard problem, and any business deploying such a gate should treat its output as a probabilistic signal rather than a verdict. The practical recommendation is to use detection as a triage tool that flags likely machine-written content for human review, not as an automatic decision-maker, and to expect performance to vary by content type.

Attribution is the harder task and should be treated as exploratory, but reaching about five times chance across eleven generators shows there is real, learnable signal in which model wrote a given text. The clearest near-term improvement is fixing the neural network's input scaling so the neural-versus-non-neural comparison is fair on both tasks.

## 10. Planned extensions for the final report

- **Fix and re-tune the neural network** with standardized inputs, then re-run the full comparison so the neural and non-neural models are evaluated on equal footing.
- **Systematic hyperparameter tuning.** The current models use default or lightly-set parameters, so the reported scores understate what each model can do. The final version will tune each model over a proper search, for example Optuna for the boosting and neural models, so the comparison reflects each model at its best.
- **A dedicated stacking ensemble** combining the linear, boosting, and neural models, with an explicit test of whether the added complexity earns a measurable lift over the best single model.

- **Character n-gram features** to capture surface artifacts the word features miss, and a per-domain breakdown of detection difficulty across the eight domains.
- **A generalization stress test (individual learning component).** The detectors are trained on generators from 2023 to 2024. To test whether they still catch current systems, the final version will generate a held-out set of text from present-day models and evaluate the detector on generators it never trained on. The design reserves a separate pool of source prompts for this held-out generation so that neither the generator nor the topic leaks into training, which keeps the generalization claim honest.

## 11. Reproducibility

Code, notebooks with visible outputs, and this report are in the project repository: https://github.com/lichris210/advancedml